

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMIZATION TASK

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	Arvind S	Reg. No.:	192525192
Section / Batch:		Date:	24/07/2026

Code of Conduct

I Arvind S / 192525192 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Arvind S

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 – 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Singly Linked List
5 marksQ7
Stack
5 marksQ8
Linked List
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

10/10 answered

* Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int key) {
4     int low = 0, high = n - 1;
5
6     while (low <= high) {
7         int mid = (low + high) / 2;
8
9         if (arr[mid] == key)
10            return mid;
11        else if (arr[mid] < key)
12            low = mid + 1;
13        else
14            high = mid - 1;
15    }
16
17    return -1;
18 }
19
20 int main() {
21     int arr[] = {10, 20, 30, 40, 50};
22     int n = 5, key = 40;
23
24     int result = binarySearch(arr, n, key);
25
26     if (result != -1)
27         printf("Element found at index %d", result);
28     else
29         printf("Element not found");
30
31     return 0;
32 }
```

Run

Save

Submit Fix

Test Input

10 20 30 40 50
40

Output

5 marks

Q4

Stack
5 marks

Q5

Stack - Balancing Symbols
5 marks

Q6

Singly Linked List
5 marks

Q7

Stack
5 marks

Q8

Linked List
5 marks

Q9

Linked List
5 marks

Q10

Queue
5 marks

10/10 answered

}

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5] = {10, 20, 30};
4 int top = 2;
5
6 int pop() {
7     if (top == -1) {
8         printf("Underflow\n");
9         return -1;
10    }
11
12    return stack[top--];
13 }
14
15 int main() {
16     int x = pop();
17
18     if (x != -1)
19         printf("Popped element = %d", x);
20
21     return 0;
22 }
```

Test Input

```
{10, 20, 30}
2
```

Output

Run

Save

Submit Fix

Stack
5 marksQ3
Stack
5 marksQ4
Stack
5 marksQ5
Stack - Balancing Symbols
5 marksQ6
Singly Linked List
5 marksQ7
Stack
5 marksQ8
Linked List
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

```
void init() {  
    top = 0;  
}
```

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int stack[MAX];  
6 int top;  
7  
8 void init() {  
9     top = -1;  
10 }  
11  
12 int main() {  
13     init();  
14  
15     printf("Top = %d", top);  
16  
17     return 0;  
18 }
```

Run

Save

Submit Fix

Test Input

stdin input (optional)

Output

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Stack - Balancing Symbols
5 marks

Q6
Singly Linked List
5 marks

Q7
Stack
5 marks

Q8
Linked List
5 marks

Q9
Linked List
5 marks

Q10
Queue
5 marks

10/10 answered

```
pop0;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 char stack[10] = {'(', '+'};  
4 char postfix[20];  
5 int top = 1, j = 0;  
6  
7 char pop() {  
8     return stack[top--];  
9 }  
10  
11 int main() {  
12     char ch = ')';  
13  
14     if (ch == ')') {  
15         while (stack[top] != '(') {  
16             postfix[j++] = pop();  
17         }  
18         pop(); // Remove '('  
19     }  
20  
21     postfix[j] = '\0';  
22     printf("Postfix = %s", postfix);  
23  
24     return 0;  
25 }
```

Test Input

Infix Expression: A+B
Current Character:)

Output

Run

Save

Submit Fix

Q2
Stack
5 marksQ3
Stack
5 marksQ4
Stack
5 marksQ5
Stack - Balancing Symbols
5 marksQ6
Singly Linked List
5 marksQ7
Stack
5 marksQ8
Linked List
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

10/10 answered

```
    } else {  
        pop();  
    }  
    return 0;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 char stack[10] = {'{'};  
4 int top = 0;  
5  
6 void pop() {  
7     top--;  
8 }  
9  
10 int main() {  
11     char ch = '}';  
12  
13     if ((ch == ']' && stack[top] == '[') ||  
14         (ch == '}' && stack[top] == '{')) {  
15  
16         pop();  
17         printf("Balanced Brackets");  
18     } else {  
19         printf("Not Balanced");  
20         return 0;  
21     }  
22  
23     return 0;  
24 }
```

Test Input

Bracket: }
Stack Top: {

Output

[Run](#) [Save](#) [Submit Fix](#)

Q4
Stack
5 marks



Q5
Stack - Balancing Symbols
5 marks



Q6
Singly Linked List
5 marks



Q7
Stack
5 marks



Q8
Linked List
5 marks



Q9
Linked List
5 marks



Q10
Queue
5 marks



10/10 answered

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *second, *third, *temp;
11
12     head = (struct Node *)malloc(sizeof(struct Node));
13     second = (struct Node *)malloc(sizeof(struct Node));
14     third = (struct Node *)malloc(sizeof(struct Node));
15
16     head->data = 10;
17     head->next = second;
18
19     second->data = 20;
20     second->next = third;
21
22     third->data = 30;
23     third->next = NULL;
24
25     temp = head;
26
27     while (temp != NULL) {
28         printf("%d ", temp->data);
29         temp = temp->next;
30     }
31
32     return 0;
33 }
```

Run

Save

Submit Fix

Test Input

10 -> 20 -> 30 -> NULL

Output



5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Stack - Balancing Symbols
5 marks

Q6

Singly Linked List
5 marks

Q7

Stack
5 marks

Q8

Linked List
5 marks

Q9

Linked List
5 marks

Q10

Queue
5 marks

10/10 answered

```
else:  
    return stack[top--]
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 int stack[5] = {10, 20, 30};  
4 int top = 2;  
5  
6 int pop() {  
7     if (top == -1) {  
8         printf("Underflow\n");  
9         return -1;  
10    } else {  
11        return stack[top--];  
12    }  
13 }  
14  
15 int main() {  
16     int item = pop();  
17  
18     if (item != -1)  
19         printf("Popped element = %d", item);  
20  
21     return 0;  
22 }
```

Run

Save

Submit Fix

Test Input

```
Stack = {10, 20, 30}  
top = 2
```

Output



Stack
5 marksQ5
Stack - Balancing Symbols
5 marksQ6
Singly Linked List
5 marksQ7
Stack
5 marksQ8
Linked List
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

10/10 answered

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *second, *third;
11     struct Node *current, *prev, *next;
12
13     head = (struct Node *)malloc(sizeof(struct Node));
14     second = (struct Node *)malloc(sizeof(struct Node));
15     third = (struct Node *)malloc(sizeof(struct Node));
16
17     head->data = 10;
18     head->next = second;
19
20     second->data = 20;
21     second->next = third;
22
23     third->data = 30;
24     third->next = NULL;
25
26     prev = NULL;
27     current = head;
28
29     while (current != NULL) {
30         next = current->next;
31         current->next = prev;
32         prev = current;
33         current = next;
34     }
35
36     head = prev;
37 }
```

Linked List: 10 -> 20 -> 30 -> NULL

Output

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head = NULL;
11     struct Node *newNode;
12
13     newNode = (struct Node *)malloc(sizeof(struct Node));
14     newNode->data = 10;
15
16     newNode->next = head;
17     head = newNode;
18
19     printf("Inserted Node = %d", head->data);
20
21     return 0;
22 }
```

Test Input

Data = 10

Output

Q3
Stack
5 marksQ4
Stack
5 marksQ5
Stack - Balancing Symbols
5 marksQ6
Singly Linked List
5 marksQ7
Stack
5 marksQ8
Linked List
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

10/10 answered

Stack
5 marks**Q5**
Stack - Balancing Symbols
5 marks**Q6**
Singly Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Your Fixed Code

```
1 #include <stdio.h>
2
3 int Q[5] = {10, 20, 30};
4 int front = 0, rear = 2;
5
6 int dequeue() {
7     if (front == -1) {
8         printf("Underflow\n");
9         return -1;
10    }
11
12    int item = Q[front++];
13
14    if (front > rear) {
15        front = rear = -1;
16    }
17
18    return item;
19 }
20
21 int main() {
22     int x = dequeue();
23
24     if (x != -1)
25         printf("Deleted element = %d", x);
26
27     return 0;
28 }
```

Submitted

Test Input

```
Queue = {10, 20, 30}
front = 0
rear = 2
```

Output

Deleted element = 10

Pending manual review

Q1
Stack
5 marks

Q2
Circular Queue
5 marks

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Doubly Linked List
5 marks

Q6
Queue
5 marks

Q7
Queue - Deque
5 marks

Q8
Binary Search
5 marks

Q9
Circular Queue
5 marks

Q1 Stack

5 marks

Point out the error?
PEEK(stack):
return stack[top+1]

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5] = {10, 20, 30};
4 int top = 2;
5
6 int peek() {
7     if (top == -1) {
8         printf("Stack Underflow\n");
9         return -1;
10    }
11    return stack[top];
12 }
13
14 int main() {
15     int item = peek();
16
17     if (item != -1)
18         printf("Top element = %d", item);
19
20     return 0;
21 }
```

Test Input

Stack = {10, 20, 30}
top = 2

Output

Run

Save

Submit Fix

5 marks

Q2

Circular Queue
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Doubly Linked List
5 marks

Q6

Queue
5 marks

Q7

Queue - Deque
5 marks

Q8

Binary Search
5 marks

Q9

Circular Queue
5 marks

Q10

Stack - Postfix Expression
5 marks

```
rear = (rear + 1) % MAX;  
queue[rear] = vehicle;  
if(rear == front){  
    printf("Queue Full");  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int queue[MAX];  
6 int front = 0, rear = 2;  
7  
8 void enqueue(int vehicle) {  
9     if ((rear + 1) % MAX == front) {  
10         printf("Queue Full\n");  
11         return;  
12     }  
13  
14     rear = (rear + 1) % MAX;  
15     queue[rear] = vehicle;  
16  
17     printf("Vehicle inserted = %d\n", vehicle);  
18 }  
19  
20 int main() {  
21     enqueue(40);  
22     return 0;  
23 }
```

Test Input

```
Vehicle = 40  
front = 0  
rear = 2
```

Output

Run

Save

Submit Fix

Q1
Stack
5 marks



Q2
Circular Queue
5 marks



Q3
Stack
5 marks



Q4
Stack
5 marks



Q5
Doubly Linked List
5 marks



Q6
Queue
5 marks



Q7
Queue - Deque
5 marks



Q8
Binary Search
5 marks



Q9
Circular Queue
5 marks



Q10
Stack - Postfix Expression
5 marks



Point out the error.

if top == MAX:

Overflow

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = 3;
7
8 void push(int item) {
9     if (top == MAX - 1) {
10         printf("Overflow\n");
11         return;
12     }
13     stack[++top] = item;
14     printf("Inserted element = %d", item);
15 }
16
17
18 int main() {
19     push(50);
20     return 0;
21 }
```

Test Input

top = 3
item = 50

Output



▶ Run

☁ Save

🔧 Submit Fix

Q2
Circular Queue
5 marks



Q3
Stack
5 marks



Q4
Stack
5 marks



Q5
Doubly Linked List
5 marks



Q6
Queue
5 marks



Q7
Queue - Deque
5 marks



Q8
Binary Search
5 marks



Q9
Circular Queue
5 marks



Q10
Stack - Postfix Expression
5 marks



10/10 answered

POP():
if top == 0 → Underflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5] = {10, 20, 30};
4 int top = 2;
5
6 int pop() {
7     if (top == -1) {
8         printf("Underflow\n");
9         return -1;
10    }
11
12    return stack[top--];
13 }
14
15 int main() {
16     int item = pop();
17
18     if (item != -1)
19         printf("Popped element = %d", item);
20
21     return 0;
22 }
```

Run

Save

Submit Fix

Test Input

Stack = {10, 20, 30}
top = 2

Output



QUESTIONS

Q1

Stack
5 marks

Q2

Circular Queue
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Doubly Linked List
5 marks

Q6

Queue
5 marks

Q7

Queue - Deque
5 marks

Q8

Binary Search
5 marks

Q9

Circular Queue
5 marks

Q10



Q5 Doubly Linked List

5 marks

Identify the error in this Doubly Linked List node deletion.

```
void deleteNode(Node* del) {  
    if (del->prev != NULL)  
        del->prev->next = del->next;  
    if (del->next != NULL)  
        del->next->prev = del->prev;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3  
4 struct Node {  
5     int data;  
6     struct Node *prev, *next;  
7 };  
8  
9 void deleteNode(struct Node **head, struct Node *del) {  
10     if (del == NULL)  
11         return;  
12  
13     if (*head == del)  
14         *head = del->next;  
15  
16     if (del->prev != NULL)  
17         del->prev->next = del->next;  
18  
19     if (del->next != NULL)  
20         del->next->prev = del->prev;
```

Test Input

Doubly Linked List: 10 <-> 20
Delete Node: 20

Output



QUESTIONS

Q1

Stack
5 marks

Q2

Circular Queue
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Doubly Linked List
5 marks

Q6

Queue
5 marks

Q7

Queue - Deque
5 marks

Q8

Binary Search
5 marks

Q9

Circular Queue
5 marks

Q10



Q6 Queue

5 marks

Identify the error in the Queue enqueue logic.

```
void enqueue(int data) {  
    if (rear == MAX) return;  
    queue[rear++] = data;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int queue[MAX];  
6 int rear = 2;  
7  
8 void enqueue(int data) {  
9     if (rear == MAX - 1) {  
10         printf("Queue Overflow\n");  
11         return;  
12     }  
13  
14     queue[++rear] = data;  
15 }  
16  
17 int main() {  
18     enqueue(40);  
19     printf("Inserted element = %d", queue[rear]);  
20  
21     return 0;  
22 }
```

Test Input

```
rear = 2  
data = 40
```

Output

Q2
Circular Queue
5 marks

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Doubly Linked List
5 marks

Q6
Queue
5 marks

Q7
Queue - Deque
5 marks

Q8
Binary Search
5 marks

Q9
Circular Queue
5 marks

Q10
Stack - Postfix Expression
5 marks

10/10 answered

```
if (front == 0) front = MAX - 1;  
else front = front - 1;  
deque[front] = data;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int deque[MAX];  
6 int front = 2;  
7  
8 void insertFront(int data) {  
9     if (front == 0)  
10         front = MAX - 1;  
11     else  
12         front = front - 1;  
13  
14     deque[front] = data;  
15 }  
16  
17 int main() {  
18     insertFront(10);  
19     printf("Inserted element = %d", deque[front]);  
20     return 0;  
21 }
```

Run

Save

Submit Fix

Test Input

```
front = 2  
data = 10
```

Output



Stack
5 marks

Q5

Doubly Linked List
5 marks

Q6

Queue
5 marks

Q7

Queue - Deque
5 marks

Q8

Binary Search
5 marks

Q9

Circular Queue
5 marks

Q10

Stack - Postfix Expression
5 marks

10/10 answered

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int l, int r, int x) {
4     if (l > r)
5         return -1;
6
7     int mid = l + (r - l) / 2;
8
9     if (arr[mid] == x)
10        return mid;
11
12    if (arr[mid] > x)
13        return binarySearch(arr, l, mid - 1, x);
14
15    return binarySearch(arr, mid + 1, r, x);
16 }
17
18 int main() {
19     int arr[] = {10, 20, 30, 40, 50};
20     int x = 40;
21
22     int result = binarySearch(arr, 0, 4, x);
23
24     if (result != -1)
25         printf("Element found at index %d", result);
26     else
27         printf("Element not found");
28
29     return 0;
30 }
```

Run

Save

Submit Fix

Test Input

Array: 10 20 30 40 50
Search Element: 40

Output

Q4

Stack
5 marks

Q5

Doubly Linked List
5 marks

Q6

Queue
5 marks

Q7

Queue - Deque
5 marks

Q8

Binary Search
5 marks

Q9

Circular Queue
5 marks

Q10

Stack - Postfix Expression
5 marks

10/10 answered

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX] = {10, 20, 30};
6 int front = 0, rear = 2;
7
8 int dequeue() {
9     if (front == -1) {
10         printf("Underflow\n");
11         return -1;
12     }
13
14     int data = queue[front];
15
16     if (front == rear)
17         front = rear = -1;
18     else
19         front = (front + 1) % MAX;
20
21     return data;
22 }
23
24 int main() {
25     int x = dequeue();
26
27     if (x != -1)
28         printf("Deleted element = %d", x);
29
30     return 0;
31 }
```

Test Input

```
Queue = {10, 20, 30}
front = 0
rear = 2
```

Output



Q5
Doubly Linked List
5 marks**Q6**
Queue
5 marks**Q7**
Queue - Deque
5 marks**Q8**
Binary Search
5 marks**Q9**
Circular Queue
5 marks**Q10**
Stack - Postfix Expression
5 marks

10/10 answered

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[10], top = -1;
4
5 void push(int x) {
6     stack[++top] = x;
7 }
8
9 int pop() {
10     return stack[top--];
11 }
12
13 int main() {
14     char expression[] = "23+";
15     int i = 0, result;
16
17     while (expression[i] != '\0') {
18         if (expression[i] >= '0' && expression[i] <= '9')
19             push(expression[i] - '0');
20         else {
21             int op2 = pop();
22             int op1 = pop();
23             result = op1 + op2;
24             push(result);
25         }
26         i++;
27     }
28
29     printf("Result = %d", pop());
30
31     return 0;
32 }
```

Test Input

Postfix Expression: 23+

Output

Result = 5

Pending manual review